1.2 Algorithms	1.2.1	be able to follow and write algorithms (flowcharts, pseudocode*, program code) that use sequence, selection, repetition (count-controlled, condition-controlled) and iteration (over every item in a data structure), and input, processing and output to solve problems
	1.2.2	understand the need for and be able to follow and write algorithms that use variables and constants and one- and two-dimensional data structures (strings, records, arrays)
	1.2.3	understand the need for and be able to follow and write algorithms that use arithmetic operators (addition, subtraction, division, multiplication, modulus, integer division, exponentiation), relational operators (equal to, less than, greater than, not equal to, less than or equal to, greater than or equal to) and logical operators (AND, OR, NOT)
	1.2.4	be able to determine the correct output of an algorithm for a given set of data and use a trace table to determine what value a variable will hold at a given point in an algorithm
	1.2.5	understand types of errors that can occur in programs (syntax, logic, runtime) and be able to identify and correct logic errors in algorithms
	1.2.6	understand how standard algorithms (bubble sort, merge sort, linear search, binary search) work

Subject content	Students should:
	1.2.7 be able to use logical reasoning and test data to evaluate an algorithm's fitness for purpose and efficiency (number of compares, number of passes through a loop, use of memory)